

GRAB: GRounding-based, Repository-Aware Boolean Gate — Reducing Hallucinations via Tool Usage Awareness in LLM-based Coding Agents

Vish Karthikeyan, Department of Computer Science at Stanford University
vk27@cs.stanford.edu

Abstract

Large language model (LLM)-based coding agents frequently hallucinate implementation details such as incorrect API calls, nonexistent file paths, and fabricated configuration keys. These failures arise when the agent attempts to act under an incomplete context window instead of explicitly requesting missing information via tools. We study the problem of *tool use awareness* for coding agents: given a prompt, a draft code snippet, and current repository context, should the agent invoke an information-gathering tool call or proceed without one? We frame this as a supervised classification problem over a repository-level hallucination feature vector, and train a constrained XGBoost classifier, which we call GRAB (GRounding-based, Repository-Aware Boolean gate), to gate tool calls within an iterative refinement loop. On SWE-bench Verified, GRAB reduces hallucination rate from 50–52% for never-tool and static baselines, and 69% for a token-level entropy-only policy, down to 44% while using approximately two tool calls per successful task, which happens to be the same tool efficiency as an always-tool upper bound. Gain-based feature importance for GRAB shows that lexical and library-grounding features from our hallucination feature vector dominate entropy-based uncertainty signals, supporting the hypothesis that repository-level grounding is a better predictor of coding hallucinations than token-level entropy alone.

1 Introduction

Large language model-based coding agents have revolutionized the software engineering industry, from elevating professional capabilities to equipping the layman with coding superpowers once reserved for seasoned developers. Particularly, these coding agents are increasingly used to automate software engineering tasks ranging from both single-function completion to repository-level development and bug fixing. Despite impressive benchmark results, such agents routinely hallucinate implementation details: a call to a nonexistent API method, a file path that does not exist in the target repository, or an import of a plausible but fabricated package. These errors are not random; they arise when the model is forced to act with incomplete repository context and does not recognize that more information is needed.

Human programmers, in contrast, resolve uncertainty by asking: they read files, inspect documentation, or run commands before committing to an implementation. Modern tool-augmented LLM agents can in principle do the same via tool calls (file reads, search, API lookups), but most systems focus on *how* to call tools rather than *when*. In this work we address the base problem in a hierarchy of tool-augmented capabilities: **when should a coding agent invoke a tool call?** And in learning to do so, what are the most important factors to consider?

We consider an iterative code refinement loop. At each step the agent receives a natural language problem statement and a current context view of the repository, generates a draft patch, and then decides whether to call a file-read oracle to expand its context before iter-

ating. Our contribution is a learned *gate* for this decision, which we call GRAB (GRounding-based, Repository-Aware Boolean gate). We:

- Build upon prior research on repository-level hallucination taxonomy [24] for coding agents and use it to construct a 9-dimensional feature vector combining grounding-based and entropy-based signals.
- Train a constrained XGBoost classifier to predict whether the current draft of generated code likely contains a *resolvable* hallucination that a tool call could fix.
- Evaluate the resulting policy on SWE-bench Verified under five conditions (never call a tool, always call a tool, static, entropy features only, and full GRAB algorithm) and measure hallucination rate, tool efficiency, and task success.

We show that a grounding-aware gating policy substantially outperforms both static heuristics and a token-level entropy-only policy, and we provide an ablation-style analysis via feature importance to determine what matters the most when learning when to ask.

2 Related Work

Hallucinations in code generation: Hallucinations by LLMs in general have been heavily studied and theorized, as evident from the literature. Huang et al [3] provide a survey of prior work on hallucinations in LLMs and distinguish epistemic uncertainty from calibration failure. Zhang et al. [24] provide a particular taxonomy

of repository-level hallucinations in code generation, including API knowledge conflicts, dependency conflicts, and non-code resource conflicts, and show that context augmentation can reduce such errors. Our work leverages this taxonomy to design features that directly target resolvable repository-level hallucinations. For ease of reference, we reproduce their taxonomy below.

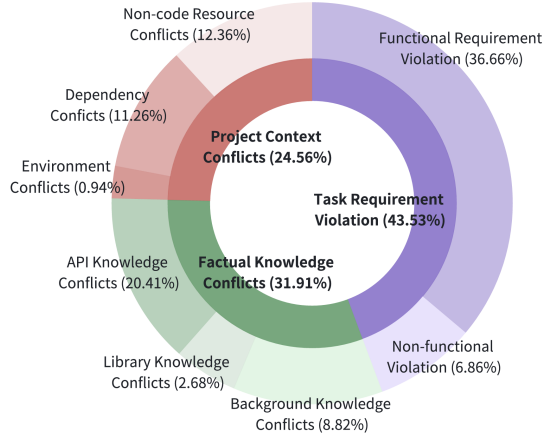


Figure 1: Taxonomy and distribution of hallucination in LLM code generation as analyzed by Zhang et al. [24]. In our work, we make the critical distinction of those hallucinations resolvable by making tool calls, and those inherent to the transformernature of LLMs.

Tool-augmented language models: Toolformer [16] and ReAct [22] show that LLMs can interleave reasoning with tool invocations. However, both treat tool call timing as an emergent property of the generation process, driven by prompt engineering or self-supervised objectives, rather than as an explicit learned policy. Meta-Tool [4] and WTU-Eval [9] benchmark when-to-use-tool behavior in general question answering, but not in code-centric and repository-level context, which are crucial for our motivation. We focus specifically on *coding agents* that operate over real repositories and study when they should call a file-read oracle.

Tool use awareness: Recent work frames tool use awareness as deciding whether a tool is needed at all given a query. We adopt this view but specialize it to repository-level code generation and pair it with a repository-aware hallucination detector. Our contribution sits at level 1 in a hierarchy of tool-augmented coding capabilities: tool use awareness, context acquisition, tool selection, tool invocation, and code generation, as elucidated below. We provide this hierarchy to ground the literature with clarity and to situate our work within the broader landscape of agentic AI research.

3 Dataset and Features

3.1 SWE-bench Verified

We evaluate on SWE-bench Verified [jimenez2024swebench], a benchmark of ~ 500 GitHub issues paired with human-validated patches across popular Python repositories (e.g. `django/django`, `pytest-dev/pytest`, `scikit-learn/scikit-learn`). Each example includes:

- `instance_id` and `repo`;
- `problem_statement`: the natural language description of the issue, used as the agent prompt;
- `patch`: the ground-truth unified diff, used only for supervision and evaluation;
- `test_patch` and `base_commit`: used to check out the correct repository state and verify completion.

We perform a deterministic 60/20/20 split with seed 42 into train/validation/test (300/100/100 examples), and we commit the resulting JSON files so all conditions share the exact same test set. For each training example, the label ($y \in \{0, 1\}$) is 1 iff the LLM’s draft (generated with empty repository context) contains at least one *resolvable* hallucination such as an undefined symbol, non-existent import, near-miss API call, or ungrounded path/env/config key according to our checker; otherwise ($y = 0$).

3.2 Hallucination-aware feature design

We adopt the repository-level taxonomy of zhang2025llmhallucinationspracticalcode, which distinguishes hallucinations arising from API knowledge, library knowledge, dependency conflicts, and non-code resources. We focus on hallucinations *resolvable by tool calls*—i.e. those where reading additional repository files can disambiguate the correct behaviour. This yields a 9-dimensional feature vector $x_i \in \mathbb{R}^9$.

API/library grounding. Let $\mathcal{A}(d)$ denote the multiset of API call sites in draft d , $\mathcal{I}$ a local API index for frequent SWE-bench libraries (`os`, `pathlib`, `requests`, `numpy`, `pandas`), $\mathcal{P}$ a cached PyPI package-name index, and $\mathcal{M}(d)$ the set of import statements. We define:

$$x^{(1)} = \frac{|\{a \in \mathcal{A}(d) : \text{sim}(a, \mathcal{I}) \geq \delta \wedge a \notin \mathcal{I}\}|}{|\mathcal{A}(d)|} \quad (1)$$

$$x^{(2)} = \frac{|\{m \in \mathcal{M}(d) : m \notin \mathcal{P}\}|}{|\mathcal{M}(d)|} \quad (2)$$

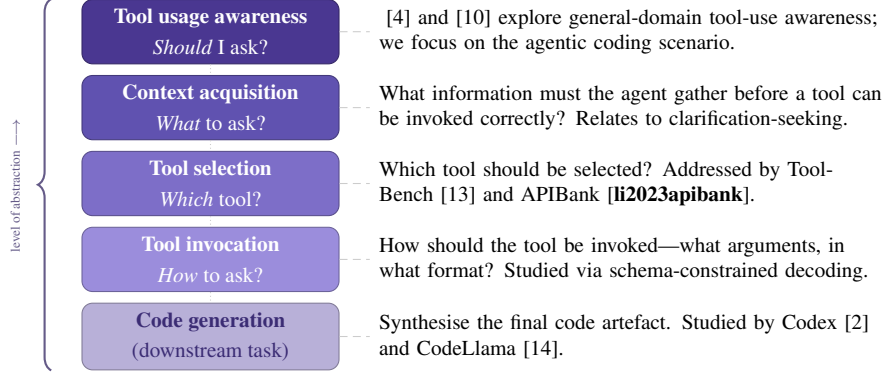


Figure 2: Hierarchy of sub-problems in tool-augmented agentic coding. We address only the highest-level sub-problem (tool usage awareness).

$$x^{(3)} = \frac{|\{a \in \mathcal{A}(d) : \mathcal{E}(a) \subseteq \mathcal{H}(d)\}|}{|\mathcal{A}(d)|} \quad (3)$$

where $x^{(1)}$ is the *lexical plausibility score*, $x^{(2)}$ the *import existence rate*, and $x^{(3)}$ the *exception-handling completeness*. Here $\mathcal{E}(a)$ is the set of documented exceptions for call site a , $\mathcal{H}(d)$ the exception types caught by `try/except` blocks in d , $\text{sim}(\cdot, \cdot)$ token-level edit similarity, and δ a fixed plausibility threshold. $x^{(1)}$ and $x^{(2)}$ are *high* when the draft is poorly grounded; $x^{(3)}$ is *high* when exceptions are well-handled.

Project context grounding. Let $\mathcal{R}(d)$ be the set of named artifacts in draft d (file paths, environment variables, config keys), and $\mathcal{R}(c_i)$ the corresponding set from context c_i :

$$x^{(4)} = \frac{|\mathcal{R}(d) \setminus \mathcal{R}(c_i)|}{|\mathcal{R}(d)|} \quad (\text{ungrounded artifact rate}) \quad (4)$$

$$x^{(5)} = \frac{|\text{Opaque}(d)|}{|\text{lines}(d)|} \quad (\text{magic value density}) \quad (5)$$

where $\text{Opaque}(d)$ is the multiset of opaque literals (hardcoded ports, timeouts, internal status codes) in d .

Entropy and context similarity. Let $\{p_t\}_{t=1}^{|d|}$ be the per-token output distributions, and $H_t = -\sum_v p_t(v) \log p_t(v)$ the Shannon entropy at position t :

$$x^{(6)} = \frac{1}{|d|} \sum_{t=1}^{|d|} H_t \quad (\text{mean token entropy}) \quad (6)$$

$$x^{(7)} = \max_t H_t \quad (\text{max token entropy}) \quad (7)$$

$$x^{(8)} = \ell_{t^*}^{(1)} - \ell_{t^*}^{(2)}, \quad t^* = \arg \max_t H_t \quad (\text{log-prob margin}) \quad (8)$$

$$x^{(9)} = \frac{\mathbf{e}(d)^\top \mathbf{e}(c_i)}{\|\mathbf{e}(d)\| \|\mathbf{e}(c_i)\|} \quad (\text{context richness}) \quad (9)$$

where $\ell_{t^*}^{(1)} \geq \ell_{t^*}^{(2)}$ are the top-two log-probabilities at the most uncertain token, and $\mathbf{e}(\cdot)$ denotes a sentence embedding. In line with prior work [huang2024metatoolbenchmarklargelanguage](#), we treat $x^{(6)}-x^{(9)}$ as *weak signals*: they measure epistemic uncertainty rather than repository-level grounding.

3.3 Label derivation

Ground-truth labels for tool-necessity are not directly available in SWE-bench. We derive them automatically from ground-truth patches using a hallucination checker. For each training task we:

1. Generate an initial draft d with empty context.
2. Compare (p, d) to the repository and ground-truth patch using the following checks:
 - Undefined symbols: names appearing in d but not in the repository or context, excluding Python keywords and builtins. We only apply this check when d parses as Python after stripping markdown fences.
 - Bad imports: imported packages not found in the cached PyPI index.
 - API mismatches: high-similarity but non-existent API calls relative to the API index.
 - Ungrounded artifacts: file paths, environment variables, and config keys not present in the prompt or repository context.
3. Assign label 1 if we detect at least one resolvable hallucination (API/library/dependency/resource), and 0 otherwise.

This procedure yields a set of (x, y) pairs for training and validation. We deliberately exclude functional and non-functional requirement violations, which are not reliably resolvable by file reads.

3.4 XGBoost training and regularization

We train two constrained XGBoost classifiers:

- A full 9-feature model used in Condition E (our GRAB gate).
- An entropy-only model on features 6–8 used in Condition D.

Because our labeled dataset is small (≈ 100 examples plus synthetic negatives), an unconstrained ensemble tends to overfit and concentrate all gain on a single feature (lexical plausibility). To mitigate this and obtain more interpretable feature importances, we adopt explicit structural constraints:

- **Shallow trees:** `max_depth` $\in \{2, 3\}$.
- **Few estimators:** `n_estimators` $\in \{50, 100, 150\}$ (full) and $\{50, 100\}$ (entropy-only).
- **Row subsampling:** `subsample` $\in \{0.7, 0.9\}$.
- **Column subsampling:** `colsample_bytree` $\in \{0.7, 1.0\}$ so different trees see different feature subsets.
- **Regularization:** `min_child_weight` $\in \{1, 5\}$ and `reg_lambda` $\in \{1.0, 5.0\}$.
- **Class imbalance:** `scale_pos_weight` = $N_{\text{neg}}/N_{\text{pos}}$.

We perform grid search over these ranges on the validation split using F1 as the selection metric. Both models use XGBoost; there is no sklearn fallback in the final experiments. We chose XGBoost because it handles heterogeneous, hand-engineered features well in the small-data regime, provides native support for missing values, and yields interpretable gain-based feature importances that let us directly test our hypothesis of whether grounding-based signals are more predictive of coding hallucinations than entropy-based uncertainty.

3.5 Experimental conditions

All conditions share the same base LLM, oracle, context-update mechanism, and test split. They differ only in the tool-call policy:

- A** Never: the agent never calls the oracle.
- B** Always: the agent always calls the oracle at each iteration until convergence.
- C** Static: a deterministic heuristic that calls the oracle if the static-analysis hallucination checker detects any undefined symbols or ungrounded artifacts.
- D** Entropy-only: the constrained XGBoost classifier trained on features 6–8 only.
- E** GRAB (ours): the constrained XGBoost classifier trained on the full 9-feature vector.

4 Experiments and Discussion

4.1 Metrics

We report three metrics on the $N = 100$ test tasks. Let d_i^* denote the final draft produced for task i , and $\mathbf{1}[\cdot]$ the indicator function.

Hallucination rate. Let $\phi(d) = 1$ if draft d contains at least one resolvable hallucination under the corrected label derivation, and 0 otherwise:

$$\hat{\mathcal{H}} = \frac{1}{N} \sum_{i=1}^N \phi(d_i^*) \quad (10)$$

Task success rate. Let G_i be the set of ground-truth added lines for task i and $\hat{G}_i$ the corresponding lines in d_i^* (up to small formatting differences). A task is considered successful when patch recall exceeds 0.5:

$$\hat{\mathcal{S}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[\frac{|G_i \cap \hat{G}_i|}{|G_i|} \geq 0.5 \right] \quad (11)$$

Tool efficiency. Let k_i be the number of tool calls made on task i . Efficiency is computed only over successful tasks:

$$\hat{\mathcal{T}} = \frac{\sum_{i=1}^N k_i \cdot \mathbf{1} \left[\frac{|G_i \cap \hat{G}_i|}{|G_i|} \geq 0.5 \right]}{\sum_{i=1}^N \mathbf{1} \left[\frac{|G_i \cap \hat{G}_i|}{|G_i|} \geq 0.5 \right]} \quad (12)$$

4.2 Main quantitative results

Table 1 summarizes the results for all five conditions.

Table 1: Results on the 100-task test set. $\hat{\mathcal{H}}$: hallucination rate (lower is better). $\hat{\mathcal{T}}$: tool efficiency, mean calls per successful task. $\hat{\mathcal{S}}$: task success rate (higher is better). Best $\hat{\mathcal{H}}$ in **bold**.

Cond.	Policy	$\hat{\mathcal{H}} \downarrow$	$\hat{\mathcal{T}}$	$\hat{\mathcal{S}} \uparrow$
A	Never	0.50	0.00	0.05
B	Always	0.48	2.00	0.05
C	Static threshold	0.52	0.00	0.05
D	XGB (entropy-only)	0.69	1.67	0.06
E	GRAB (ours)	0.44	2.00	0.05

The most striking result in Table 1 is Condition D: an entropy-only gating policy yields a *higher* hallucination

rate (69%) than even the never-tool baseline (A, 50%) and the static heuristic (C, 52%), despite using a comparable number of tool calls. This confirms that token-level entropy, by itself, is a poor predictor of coding hallucinations in this setting.

In contrast, our full grounding-aware policy GRAB (Condition E) achieves the lowest hallucination rate at 44%, improving over all baselines (A–D) while using approximately the same number of tool calls per successful task as the always-tool upper bound (B: 2.00 vs. E: 2.00). This suggests that repository-level grounding features like API and library knowledge, import validity, and exception handling are far more informative for tool-use awareness than generation-side uncertainty alone.

Task success rates are low and relatively flat (5–6%) across conditions under our strict patch-recall metric. This is partly due to the difficulty of SWE-bench Verified and our small training set; the primary effect of tool calls in our setup is to reduce hallucinations, not to dramatically increase exact patch recall. Note that hallucination rate and task success measure different axes of performance: hallucination rate is a token-/symbol-level quality metric (e.g., undefined symbols, bogus imports, API mismatches), whereas task success requires reconstructing the full ground-truth patch. Under our setting, many drafts become more grounded without reaching exact patch reconstruction, so the two metrics are only loosely coupled.

4.3 Feature importance analysis

To understand which features drive the full algorithm, we extract gain-based feature importance scores from the constrained XGBoost model used in Condition E. The normalized gains are:

- Import existence: $\approx 56\%$
- Lexical plausibility: $\approx 31\%$
- Exception-handling completeness: $\approx 13\%$
- All other features (ungrounded artifact rate, magic value density, mean/max entropy, log-prob margin, context richness): $\approx 0\%$

This distribution is strikingly aligned with our design: the model relies almost entirely on library and API grounding features, with entropy- and context-based features contributing negligibly. In earlier unconstrained runs (not reported), the model collapsed onto lexical plausibility alone; the added structural regularization (shallow trees, subsampling, L2) encourages the ensemble to exploit multiple, complementary grounding features without overfitting to a single axis.

4.4 Entropy-only policy

Condition D uses a constrained XGBoost classifier trained on entropy features only (mean/max entropy and log-prob margin). Despite being trained with the same procedure as E, D produces a significantly higher hallucination rate (69%) than both no-tool baselines and GRAB. This reinforces recent findings that token-level entropy is poorly calibrated for hallucination in code and that uncertainty signals must be grounded in repository structure and dependencies to be useful.

5 Limitations and future work

First, we train GRAB on a relatively small number of labeled trajectories from the SWE-bench Verified train split, which constrains both model capacity and the reliability of feature-importance estimates. Second, our evaluation is Python-only and relies on a strict patch-recall success metric, so the observed improvements in hallucination rate do not always translate into large gains in exact patch reconstruction. Finally, we focus exclusively on when to call a single oracle file-read tool, holding tool selection and formulation fixed. Future work includes scaling GRAB to larger, multilingual codebases, enriching the tool set (e.g., tests, static analyzers, documentation search), exploring softer or ranking-based success metrics, and jointly learning timing, selection, and query formulation within a unified tool-use policy.

6 Conclusion

We introduced GRAB, a constrained XGBoost-based, repository-aware tool-call gating policy for LLM-based coding agents, learned over a feature vector derived from a hallucination taxonomy.

1. On SWE-bench Verified, GRAB reduces hallucination rate relative to never-tool and static baselines, while an entropy-only variant substantially underperforms.
2. Gain-based feature importance shows that grounding-based features (import existence, lexical plausibility, and exception handling completeness) dominate entropy in predicting hallucinations.
3. This supports the view that coding agents should reason about repository structure rather than token-level uncertainty alone when deciding whether to ask for help.

References

- [1] Vibhor Agarwal et al. *CodeMirage: Hallucinations in Code Generated by Large Language Models*. 2025. arXiv: 2408.08333 [cs.SE]. URL: <https://arxiv.org/abs/2408.08333>.
- [2] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG].
- [3] Lei Huang et al. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *ACM Transactions on Information Systems* 43.2 (Jan. 2025), pp. 1–55. ISSN: 1558-2868. DOI: 10.1145/3703155. URL: <http://dx.doi.org/10.1145/3703155>.
- [4] Yue Huang et al. *MetaTool Benchmark for Large Language Models: Deciding Whether to Use Tools and Which to Use*. 2024. arXiv: 2310.03128 [cs.SE]. URL: <https://arxiv.org/abs/2310.03128>.
- [5] Alon Jacovi et al. *A Comprehensive Evaluation of Tool-Assisted Generation Strategies*. 2023. arXiv: 2310.10062 [cs.CL]. URL: <https://arxiv.org/abs/2310.10062>.
- [6] Nihal Jain et al. *On Mitigating Code LLM Hallucinations with API Documentation*. 2024. arXiv: 2407.09726 [cs.CL]. URL: <https://arxiv.org/abs/2407.09726>.
- [7] Guanyu Jiang et al. *XSkill: Continual Learning from Experience and Skills in Multimodal Agents*. 2026. arXiv: 2603.12056 [cs.AI]. URL: <https://arxiv.org/abs/2603.12056>.
- [8] Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. *SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Large Language Models*. 2023. arXiv: 2303.08896 [cs.CL]. URL: <https://arxiv.org/abs/2303.08896>.
- [9] Kangyun Ning et al. *WTU-EVAL: A Whether-or-Not Tool Usage Evaluation Benchmark for Large Language Models*. 2024. arXiv: 2407.12823 [cs.CL]. URL: <https://arxiv.org/abs/2407.12823>.
- [10] Kangyun Ning et al. *WTU-EVAL: A Whether-or-Not Tool Usage Evaluation Benchmark for Large Language Models*. 2024. arXiv: 2407.12823 [cs.CL].
- [11] Aaron Parisi, Yao Zhao, and Noah Fiedel. *TALM: Tool Augmented Language Models*. 2022. arXiv: 2205.12255 [cs.CL]. URL: <https://arxiv.org/abs/2205.12255>.
- [12] Marc Pavel et al. “Hallucination in LLM-Based Code Generation: An Automotive Case Study”. In: *2025 3rd International Conference on Foundation and Large Language Models (FLLM)*. 2025, pp. 101–106. DOI: 10.1109/FLLM67465.2025.11391125.
- [13] Yujia Qin et al. *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs*. 2023. arXiv: 2307.16789 [cs.AI].
- [14] Baptiste Rozière et al. *Code Llama: Open Foundation Models for Code*. 2023. arXiv: 2308.12950 [cs.PL].
- [15] Timo Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. 2023. arXiv: 2302.04761 [cs.CL]. URL: <https://arxiv.org/abs/2302.04761>.
- [16] Timo Schick et al. “Toolformer: Language Models Can Teach Themselves to Use Tools”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023. arXiv: 2302.04761 [cs.CL].
- [17] Weijia Shi et al. *Trusting Your Evidence: Hallucinate Less with Context-aware Decoding*. 2023. arXiv: 2305.14739 [cs.CL]. URL: <https://arxiv.org/abs/2305.14739>.
- [18] Chong Wang et al. *Teaching Code LLMs to Use Autocompletion Tools in Repository-Level Code Generation*. 2024. arXiv: 2401.06391 [cs.SE]. URL: <https://arxiv.org/abs/2401.06391>.
- [19] Yanlin Wang et al. *Beyond Functional Correctness: Investigating Coding Style Inconsistencies in Large Language Models*. 2025. arXiv: 2407.00456 [cs.SE]. URL: <https://arxiv.org/abs/2407.00456>.
- [20] Pengyue Yang et al. *Trust in One Round: Confidence Estimation for Large Language Models via Structural Signals*. 2026. arXiv: 2602.00977 [cs.CL]. URL: <https://arxiv.org/abs/2602.00977>.
- [21] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. arXiv: 2210.03629 [cs.CL]. URL: <https://arxiv.org/abs/2210.03629>.
- [22] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *International Conference on Learning Representations (ICLR)*. 2023. arXiv: 2210.03629 [cs.CL].
- [23] Fengji Zhang et al. *RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation*. 2023. arXiv: 2303.12570 [cs.CL]. URL: <https://arxiv.org/abs/2303.12570>.
- [24] Ziyao Zhang et al. *LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation*. 2025. arXiv: 2409.20550 [cs.SE]. URL: <https://arxiv.org/abs/2409.20550>.